AutoDSC: An Automated Data Stream Clustering Framework

Anonymous

***Abstract*—Data stream clustering faces unique challenges in dynamic environments where data distributions evolve continuously and computational resources are limited. While AutoML has transformed supervised learning, its application to unsupervised streaming tasks remains largely unexplored. Traditional clustering algorithms are sensitive to algorithm selection, hyperparameter configuration, and changes in data characteristics. We introduce AutoDSC (Automated Data Stream Clustering), a novel framework that automatically optimizes algorithm selection and hyperparameter tuning for real-time stream clustering. AutoDSC employs Population-Guided Stochastic Search Optimization (PGSSO), leveraging elite configurations to guide efficient parameter exploration while dynamically adjusting exploration rates based on performance feedback. The framework incorporates drift-aware adaptation for a proactive response to distributional changes and multi-objective evaluation with clustering validation indices for robust performance assessment. Our theoretical analysis establishes regret bounds for the optimization process. Comprehensive experiments demonstrate that AutoDSC achieves statistically significant improvements in clustering quality while maintaining computational efficiency compared to existing automated approaches.**

***Index Terms*—data stream clustering, AutoML, concept drift, hyperparameter optimization, online learning**

I. Introduction

Automated Machine Learning (AutoML) has significantly advanced the accuracy and efficiency of machine learning workflows by automating tasks such as algorithm selection, hyperparameter optimization, and pipeline synthesis [1]. While AutoML has seen widespread adoption in supervised learning, its application to clustering remains comparatively less explored, mainly due to the absence of a universally applicable objective function for evaluation [2]. Recent AutoML solutions for clustering include AutoML4Clust [3], AutoCluster [4], cSmartML [5], ML2DAC [6], TPE-AutoClust [7], and PoAC [8]. These approaches address clustering challenges using various automated methodologies such as meta-learning, pipeline composition, and problem-specific heuristics.

However, with the increasing availability of streaming data from sources like Internet of Things (IoT) sensors, web traffic, and social media, traditional static AutoML approaches are insufficient [9]. They cannot adapt to dynamic environments or process data in a single pass due to concept drift and memory limitations [10]. Existing online AutoML frameworks such as Online AutoML [11], ASML [12], ChaCha [13], and Auto-Reg [14] primarily focus on supervised learning tasks. Meanwhile, unsupervised learning, particularly clustering, requires specialized AutoML solutions. At present, ConfStream [15] and OnlineAutoClust [16] represent the only established AutoML solutions for stream clustering. Nevertheless, they suffer from significant limitations. ConfStream relies on single clustering evaluation metrics, and OnlineAutoClust uses computationally intensive offline search methods, which are inefficient for real-time streaming scenarios. These limitations highlight the critical need for tailored AutoML solutions designed specifically for data stream clustering [17]. This paper introduces AutoDSC (Automated Data Stream Clustering), a comprehensive AutoML framework for unsupervised data stream clustering. Our key contributions are:

- We propose AutoDSC, the first AutoML framework to unify elite-guided parameter optimization, concept drift detection, and multi-objective clustering evaluation in a single practical framework, introducing Population-Guided Stochastic Search Optimization (PGSSO) as a novel optimization mechanism that dynamically balances exploration and exploitation based on performance feedback in non-stationary streaming environments.
- We establish formal regret bounds proving convergence for the optimization process and demonstrate efficient recovery after concept drift detection.
- Through extensive experiments on synthetic and real-world data streams, we demonstrate that AutoDSC achieves statistically significant improvements in clustering quality while maintaining computational efficiency.

II. Related Work

Data stream clustering addresses the challenges of processing continuous, high-velocity data with limited computational resources [10], [18]. CluStream [19] pioneered the micro-cluster approach using pyramidal time frames to maintain summary statistics for scalable analysis. DenStream [20] advanced density-based clustering by introducing core-micro-clusters, potential micro-clusters, and outlier micro-clusters. DBSTREAM [21] enhanced density estimation by incorporating shared density graphs that explicitly model inter-micro-cluster relationships. This provides better resilience to concept drift compared to DenStream. For k-means-based approaches [22], mini-batch optimization combined with sparse projections has been developed to enable web-scale clustering with significantly reduced latency for real-time streaming applications. STREAMKMeans [23] is an alternative version of the stream local search [24] that replaces k-medians with incremental k-means.

Recent developments have focused on automated algorithm selection and hyperparameter optimization (CASH) [1] for stream clustering. ConfStream [15] represents an early attempt

at automated stream clustering configuration, using an ensemble of different clustering algorithms and configurations that dynamically adapt by evaluating clustering quality and sampling nearby new configurations proportionally to their performance using a normal distribution. However, ConfStream has significant limitations, including its reliance on a single Silhouette Width metric for quality assessment and limited parameter search capabilities. OnlineAutoClust [16] developed an AutoML framework that combines Bayesian optimization with clustering ensembles, employing Tree-structured Parzen Estimators (TPE) for algorithm selection and hyperparameter tuning. However, OnlineAutoClust suffers from critical limitations, including dependence on offline tuning methods, high computational requirements, and sensitivity to allocated time budgets, which can lead to suboptimal pipeline selections during rapid concept drift. For concept drift detection, Dynamic Adapting Window Independence Drift Detection (DAWIDD) [25] addresses drift through non-parametric independence tests between random variables, providing robust detection across diverse drift types without assuming underlying models.

III. Automated Data Stream Clustering

We introduce Automated Data Stream Clustering (AutoDSC), a novel AutoML framework tailored for non-stationary data stream clustering. AutoDSC processes data in fixed-size windows. At the end of each window, AutoDSC evaluates clustering configurations using multiple normalized validation indices. The framework maintains an elite population of high-performing configurations to guide future parameter exploration via PGSSO. The framework adaptively balances exploration and exploitation through dynamic rate adjustment based on performance feedback. To quickly adapt to distributional changes, AutoDSC employs DAWIDD drift detection that triggers an internal state reset when concept drift is detected. Figure 1 illustrates the high-level workflow of AutoDSC, while Algorithm 1 details the complete operational steps.

AutoDSC processes an infinite data stream $\mathcal{S} = \{x_1, x_2, \ldots, x_t, \ldots\}$ where each instance $x_t \in \mathbb{R}^d$ arrives at timestamp t through a fixed-size windowing mechanism $W_t = \{x_{t-w+1}, x_{t-w+2}, \ldots, x_t\}$ with capacity w. The framework maintains a configuration population $\mathcal{A}_t$ containing multiple clustering algorithms with different hyperparameter settings, continuously adapts these configurations based on performance feedback, and provides real-time predictions using the best-performing configuration. The configuration space $\Theta = \prod_{i=1}^{|\mathcal{A}|} \Theta_i$ encompasses parameter spaces across all algorithms in the portfolio $\mathcal{A}$, where each configuration $\theta = (a_i, \phi_i)$ specifies both algorithm selection $a_i \in \mathcal{A}$ and hyperparameter values $\phi_i \in \Theta_i$. As each new data point arrives, the system performs incremental updates across all configurations:

$$\forall \theta_i \in \mathcal{A}_t : \quad M_i^{(t)} = \text{LearnOne}(M_i^{(t-1)}, x_t) \tag{1}$$

where $M_i^{(t)}$ represents the model state of configuration θ_i at time t. When the window reaches capacity, the framework initiates evaluation and optimization operations.

AutoDSC first performs concept drift detection using DAWIDD [25] by extracting statistical features from each window $\mathbf{f}_t \leftarrow [\mu_{\|x\|}, \sigma_{\|x\|}, \min_{\|x\|}, \max_{\|x\|}, \text{median}_{\|x\|}, t]^T$. If significant distributional changes are detected (p-value $< \tau$), the system resets its internal state and increases exploration to rapidly adapt to the new data regime.

For performance assessment, AutoDSC evaluates clustering quality using three normalized clustering validation indices: Normalized Silhouette Coefficient NSC $= (1 + \text{SC})/2$, Normalized Davies-Bouldin Index NDBI $= 1 - \tanh(\text{DBI})$, and Normalized Calinski-Harabasz Index NCHI $= \tanh(\text{CHI})$, ensuring all metrics are bounded to $[0, 1]$ where higher values indicate better clustering quality. The expected clustering quality for configuration θ at time t is:

$$\mu_t(\theta) = \frac{1}{3}[\text{NSC} + \text{NDBI} + \text{NCHI}] \tag{2}$$

AutoDSC dynamically adjusts exploration behavior based on performance improvement trends. The exploration rate α_t adapts based on relative improvement:

$$\Delta\mu_t = \mu_t(\theta_t^*) - \mu_{t-1}(\theta_{t-1}^*) \tag{3}$$

$$\alpha_{t+1} = \begin{cases} \max(0.1, \alpha_t \cdot (1 - \gamma \cdot r_t)) & \text{if } \Delta\mu_t > 0 \\ \min(1.0, \alpha_t \cdot (1 + \gamma \cdot |r_t|)) & \text{if } \Delta\mu_t \leq 0 \end{cases} \tag{4}$$

where r_t represents normalized relative improvement and γ controls adaptation rate. This mechanism decreases exploration when performance improves (favoring exploitation) and increases exploration when performance stagnates (encouraging broader search).

The framework employs **Population-Guided Stochastic Search Optimization (PGSSO)** (Algorithm 2) to jointly optimize algorithm selection and hyperparameter tuning. PGSSO maintains an elite population $\mathcal{E}_t = \{(\theta_i, s_i)\}_{i=1}^{e}$ where each configuration $\theta_i = (a_i, \phi_i)$ pairs an algorithm choice with its hyperparameters, and $s_i = \mu_t(\theta_i)$ represents its performance score. By tracking which algorithm-hyperparameter combinations perform best, PGSSO naturally learns both which algorithms suit current data characteristics and their optimal settings. The elite population encodes this joint preference: frequently appearing algorithms indicate superior algorithmic choices, while their parameter distributions reveal effective settings. PGSSO adaptively balances exploration and exploitation through dynamic rate α_t, computing statistical distributions over both algorithm selections (via frequencies) and their parameters (via means μ_j and variances σ_j^2), which together guide sampling of new promising algorithm-hyperparameter configurations.

PGSSO generates new configurations through probabilistic sampling:

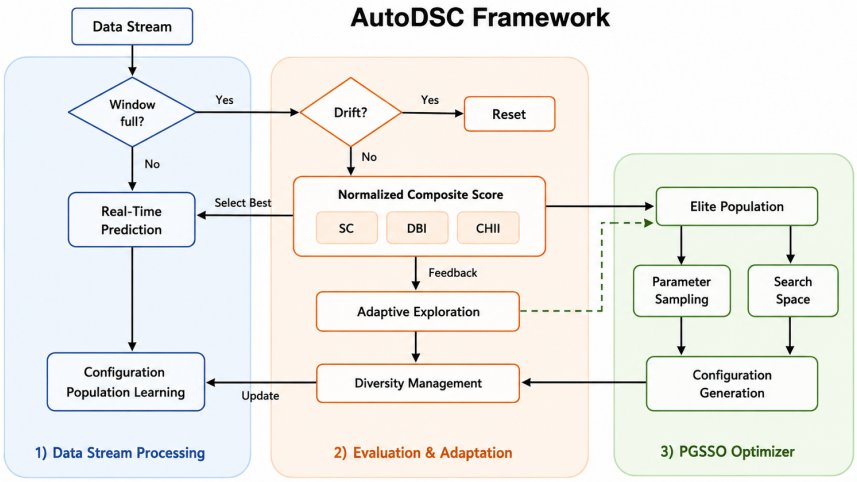


Fig. 1. AutoDSC Framework Overview

$$\theta_{\text{new}} = \begin{cases} \text{Elite-Guided}(\mathcal{E}_t) & \text{with probability } (1-\alpha_t) \\ \text{Random}(\Theta) & \text{with probability } \alpha_t \end{cases} \tag{5}$$

Elite-guided sampling first selects an algorithm based on elite frequencies, then samples its hyperparameters from learned distributions. Algorithm selection uses:

$$P(a = a_k) = \frac{\sum_{(\theta,s)\in\mathcal{E}_t} \mathbf{I}[a = a_k]}{|\mathcal{E}_t|} \tag{6}$$

where $a_k \in \mathcal{A}$ represents available algorithms. Given selected algorithm a_k, numerical hyperparameters use Gaussian distributions from elite configurations of that algorithm:

$$\phi_j \sim \mathcal{N}(\mu_j, \sigma_j^2) \text{ where } \mu_j = \frac{1}{|\mathcal{E}_t^{a_k}|} \sum_{(\theta,s)\in\mathcal{E}_t^{a_k}} \phi_j \tag{7}$$

and $\mathcal{E}_t^{a_k} = \{(\theta, s) \in \mathcal{E}_t : a = a_k\}$ contains elite configurations using algorithm a_k. Categorical hyperparameters similarly use frequency-based sampling within algorithm-specific elites. Figure 2 illustrates this hierarchical sampling process.

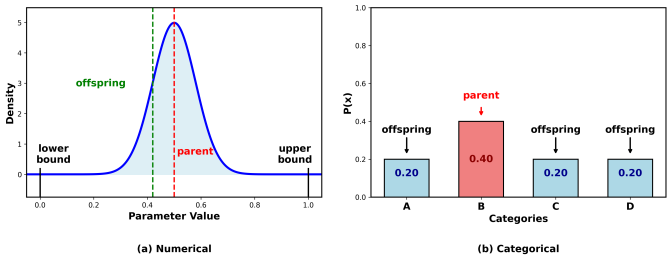


Fig. 2. Parameter Sampling Visualization.

Population diversity is maintained by computing configuration distances:

$$d(\theta_1, \theta_2) = \begin{cases} 1.0 & \text{if algorithms differ} \\ \frac{1}{|\mathcal{P}|} \sum_{p\in\mathcal{P}} d_p(\theta_1[p], \theta_2[p]) & \text{otherwise} \end{cases} \tag{8}$$

where $\mathcal{P}$ represents common parameters and d_p computes normalized parameter-specific distances. New configurations are accepted only if they maintain a minimum pairwise diversity τ_{div} with existing population members.

Throughout the process, AutoDSC provides real-time clustering predictions using the incumbent (best) configuration:

$$\theta_t^* = \arg\max_{\theta_i\in\mathcal{A}_t} \mu_t(\theta_i) \tag{9}$$

A. Computational Complexity

The framework's computational complexity scales efficiently with population size and window parameters. Per-instance processing requires $O(K \cdot C)$ operations where $K = |\mathcal{A}_t|$ and C denotes average algorithm complexity. Window-end evaluation involves $O(K \cdot \rho w \cdot d)$ operations for evaluation on ρ-sampled data plus $O(K \cdot n_{\theta,t})$ for updating evaluation counts. Total per-window complexity is $O(K \cdot \rho w \cdot d + K^2)$, where K^2 accounts for population management. Space complexity is $O((K + e_{\max}) \cdot |\Theta| + w \cdot d)$.

IV. Theoretical Analysis

We analyze AutoDSC using a phase-based approach where phases correspond to periods with fixed incumbent configurations. Let $j \in \{1, ..., T/w\}$ index windows where T/w represents the total number of windows given T instances and window size w, and $t \in \{1, ..., T\}$ index instances. We analyze AutoDSC's performance over T/w windows divided into $M + 1$ phases, where phase m spans N_m windows from window j_m to $j_{m+1} - 1$ with population $\mathcal{A}_{j_m}$ and incumbent configuration $\theta^*_{j_m}$. We track cumulative evaluations $n_{\theta,j}$ for each configuration θ across all windows up to window j. The

Algorithm 1 AutoDSC

Input: Data stream $\mathcal{S} = \{x_1, x_2, \ldots\}$, $x_t \in \mathbb{R}^d$
Parameter: Window size w, Population size K, Diversity threshold τ_{div}, Adaptation rate γ
Output: Cluster predictions $\{y_t\}_{t=1}^{\infty}$ using θ_t^*

$e_{\max} \leftarrow \lfloor K/2 \rfloor$, $\rho \leftarrow 0.25$
$\mathcal{E}_0 \leftarrow \emptyset$, $\alpha_0 \leftarrow 0.5$
$\mathcal{A}_0 \leftarrow \{\theta_i\}_{i=1}^{K}$ where $\theta_i \sim \text{Uniform}(\Theta)$
$W_0 \leftarrow \emptyset$, $\theta_0^* \leftarrow \emptyset$
$\mu_{-1}(\theta_{-1}^*) \leftarrow -\infty$
for each $x_t \in \mathcal{S}$ **do**
  $W_t \leftarrow W_{t-1} \cup \{x_t\}$
  for $\theta_i \in \mathcal{A}_t$ **do**
    $M_i^{(t)} \leftarrow \text{LEARNONE}(M_i^{(t-1)}, x_t)$
  end for
  if $|W_t| = w$ **then**
    $\mathbf{f}_t \leftarrow [\mu_{\|x\|}, \sigma_{\|x\|}, \min_{\|x\|}, \max_{\|x\|}, \text{median}_{\|x\|}, t]^T$
    if DAWIDD$(\mathbf{f}_t) =$ drift **then**
      RESET()
      continue
    end if
    $S_t \leftarrow \text{SAMPLE}(W_t, \rho)$
    for $\theta_i \in \mathcal{A}_t$ **do**
      $L_i \leftarrow \{\text{PREDICTONE}(M_i, s_j) : s_j \in S_t\}$
      $\mu_t(\theta_i) \leftarrow \text{NORMCVI}(S_t, L_i)$
      $n_{\theta_i,t} \leftarrow n_{\theta_i,t-1} + 1$
    end for
    $\mu_{\text{best}}^{(t)} \leftarrow \max_i \mu_t(\theta_i)$
    $\alpha_{t+1} \leftarrow \text{ADAPT}(\alpha_t, \mu_{t-1}(\theta_{t-1}^*), \mu_{\text{best}}^{(t)}, \gamma)$
    $\mathcal{C}_{\text{new}} \leftarrow \text{PGSSO}(\{(\theta_i, \mu_t(\theta_i))\}, \alpha_{t+1}, \Theta)$
    $\mathcal{A}_{t+1} \leftarrow \text{SELECTDIVERSE}(\mathcal{A}_t \cup \mathcal{C}_{\text{new}}, K, \tau_{\text{div}})$
    $\theta_t^* \leftarrow \arg\max_{\theta_i \in \mathcal{A}_{t+1}} \mu_t(\theta_i)$
    $W_t \leftarrow \emptyset$
  end if
end for

Algorithm 2 PGSSO

Input: Current configurations with scores $\{(\theta_i, s_i)\}$
Parameter: Exploration rate $\alpha \in [0, 1]$, Parameter space Θ
Output: New configuration set $\mathcal{C}_{\text{new}}$

$\mathcal{E} \leftarrow \text{UPDATEELITEPOPULATION}(\{(\theta_i, s_i)\}, e_{\max})$
$\mathcal{E} \leftarrow \text{UPDATEPARAMETERDISTRIBUTIONS}(\mathcal{E})$
$\mathcal{C}_{\text{new}} \leftarrow \emptyset$
for $j = 1$ to n_{gen} **do**
  $u \leftarrow \text{Uniform}(0, 1)$
  if $u > \alpha$ **then**
    $\theta_{\text{new}} \leftarrow \text{SAMPLEFROMELITE}(\mathcal{E}, \Theta)$
  else
    $\theta_{\text{new}} \leftarrow \text{RANDOMSAMPLE}(\Theta)$
  end if
  $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\theta_{\text{new}}\}$
end for
return $\mathcal{C}_{\text{new}}$

expected clustering quality for configuration θ at window j is defined as:

$$\mu_j(\theta) = \mathbb{E}[r_j(\theta)] \tag{10}$$

where $r_j(\theta) \in [0, 1]$ represents the instantaneous composite CVI score, and $\theta_j^{\text{opt}} = \arg\max_{\theta \in \Theta} \mu_j(\theta)$ denotes the optimal configuration at window j.

Definition 1: Let $p \in (0, 1)$ denote the sample complexity parameter where estimation error decreases as $O(n^{-p})$ with n samples (typically $p = 1/2$ for clustering).

Definition 2: Let comp_θ denote the computational complexity of evaluating configuration θ on one window.

Assumption 1: Rewards are bounded: $r_j(\theta) \in [0, 1]$.

Assumption 2: Within stationary segments, $\mu_j(\theta)$ is L-Lipschitz in θ.

Assumption 3: The optimal configuration θ_j^{opt} enters the population within $O(\log |\Theta|)$ windows after each concept drift.

Define confidence intervals:

$$\epsilon_{\theta,j} = O\left(\sqrt{\frac{\log(|\Theta| j/\delta)}{n_{\theta,j}}}\right) \tag{11}$$

where $n_{\theta,j}$ is the number of evaluations of θ by window j and $\delta \in (0, 1)$ is a confidence parameter [26].

Lemma 1 (PGSSO Selection Properties): With probability $\geq 1 - \delta$ [27]:

Claim 1: If $\mu_j(\theta) - \mu_j(\theta_{j_m}^*) + 2\epsilon_{\theta,j} + 3\epsilon_{\theta_{j_m}^*,j} > 0$, then PGSSO selects θ as the new incumbent $\theta_{j_{m+1}}^*$.

Claim 2: When the incumbent switches from $\theta_{j_m}^*$ to $\theta_{j_{m+1}}^*$, we have $\mu_{j_{m+1}}(\theta_{j_m}^*) - \mu_{j_{m+1}}(\theta_{j_{m+1}}^*) > \epsilon_{\theta_{j_m}^*,j_{m+1}}$.

Claim 3: If $\mu_j(\theta) < \mu_j(\theta_{j_m}^*)$, then θ will not replace $\theta_{j_m}^*$ as the incumbent.

Lemma 2 (Drift Detection): DAWIDD detects concept drift within $O(\log T)$ windows with probability $\geq 1 - \delta$, triggering population reset.

Lemma 3 (Adaptive Exploration): The adaptive exploration mechanism in PGSSO, which adjusts α_j based on performance trends, improves the expected regret constants by a factor of $(1+\gamma)^{-1}$ compared to fixed exploration, where γ is the adaptation rate.

Proposition 1 (Window Regret): With sample complexity parameter p and properly set confidence intervals, AutoDSC achieves:

$$\sum_{m=0}^{M} \sum_{j=j_m}^{j_{m+1}-1} \left(\mu_j(\theta_j^{\text{opt}}) - \mu_j(\theta_{j_m}^*)\right) = \tilde{O}\left(\frac{K \cdot \text{comp}_{\theta^{\text{opt}}} \cdot (T/w)^p}{p(1+\gamma)} + \text{comp}_{\theta^*} (T/w)^{\frac{1}{2-p}}\right) \tag{12}$$

where $K = \max_j |\mathcal{A}_j|$ is the maximum population size.

Proof Sketch: The bound decomposes regret into two terms: (i) regret from suboptimal population sampling, bounded by PGSSO's exploration guarantee with adaptive exploration reducing the constant by $(1 + \gamma)^{-1}$, and (ii) regret from incumbent updates. Lemma 1 ensures incumbent

switches occur only when sufficiently beneficial, limiting the number of phases $M = O((T/w)^{\frac{1-p}{2-p}})$. The confidence intervals shrink as $O(n_{\theta,j}^{-1/2})$, yielding the stated bounds [26].

Theorem 1 (Total Regret): Under the same conditions, AutoDSC achieves instance-level regret:

$$\sum_{t=1}^{T} \left(\mu_{\lceil t/w \rceil}(\theta^{\text{opt}}_{\lceil t/w \rceil}) - \mu_{\lceil t/w \rceil}(\theta^{*}_{\lceil t/w \rceil}) \right) = \tilde{O}\left(\frac{K \cdot \text{comp}_{\theta^{\text{opt}}} \cdot T^{p}}{(1+\gamma)} + \text{comp}_{\theta^{*}} T^{\frac{1}{2-p}} \right) \quad (13)$$

Proof Sketch: Since clustering quality is constant within windows, summing over T instances with window-level quality yields:

$$\sum_{t=1}^{T}(\cdot) = \sum_{j=1}^{T/w} w \cdot (\mu_j(\theta_j^{\text{opt}}) - \mu_j(\theta_j^{*})) \quad (14)$$

Applying Proposition 1 and noting w is constant gives the result.

Corollary 1 (Drift Recovery): After concept drift, AutoDSC recovers ϵ-optimal performance within $O(\log|\Theta|/\epsilon^2)$ windows.

For $p = 1/2$, the regret bound becomes $\tilde{O}(\sqrt{T})$, matching optimal rates for non-stationary bandits with concept drift [28].

V. Experimental Setup

Our experimental design follows rigorous evaluation protocols to ensure reproducible and statistically valid results.[1]

Datasets: We selected 12 datasets comprising 6 real-world and 6 synthetic streams with varying sizes, dimensionalities, and cluster counts, as summarized in Table I.

TABLE I
Summary of Datasets

Dataset Name	Type	Instances	Attributes	Clusters
Adult	Real	48,842	14	4
Electricity	Real	45,312	8	2
Forest Cover	Real	581,012	54	7
Insects	Real	52,848	33	6
Airlines	Real	539,383	7	2
Vehicle	Real	98,528	100	3
Hyperplane	Synth	500,000	10	2
Moving (RBF)	Synth	200,000	10	5
Moving Squares	Synth	200,000	2	4
SEA (Abrupt)	Synth	500,000	3	2
SEA (Mixed)	Synth	500,000	3	2
Random (RBF)	Synth	100,000	4	4

Evaluation Protocol: All experiments use prequential evaluation (test-then-train) [10] with 10 independent runs using random seeds 1-10 to account for stochastic variability. Performance is measured using the non-adjusted Rand Index [29], where ground truth labels are used solely for evaluation, not training or model selection. We also record execution time (minutes) and memory usage (MB) to assess computational efficiency. Statistical significance is assessed using the Friedman test followed by the Nemenyi post-hoc test [30] with $\alpha = 0.10$.

Baseline Methods: We compare AutoDSC against four baseline approaches. *Default(Best)* represents the best-performing algorithm among CluStream, DenStream, DBSTREAM, KMeans, and STREAMKMeans using default parameters, selected per dataset based on mean Rand Index across 10 runs. This approach reflects common practice among non-expert users who apply standard algorithms without extensive hyperparameter tuning. *ConfStream* [15] and *OnlineAutoClust* [16] represent established AutoML approaches for stream clustering that optimize hyperparameters within the same parameter ranges. *RandomSearch* [31] operates similarly to AutoDSC's framework structure but uses purely random sampling for algorithm selection and hyperparameter configuration generation, providing a direct comparison to our guided optimization approach.

Configuration: AutoDSC uses the parameters specified in the methodology: population size 20, window size 1000, exploration rate 0.5, adaptation rate $\gamma = 0.1$, and diversity threshold 0.1. For fair comparison, following standard AutoML benchmarking practices [32], all methods optimize hyperparameters within identical search spaces as specified in Table II. Other baseline-specific parameters remain as reported in their original papers.

All experiments were conducted on AMD EPYC 7702 64-core servers with 1TB RAM, utilizing parallelization for computational efficiency. All clustering algorithms were implemented using the River Python library [23] for online machine learning.

VI. Results Analysis

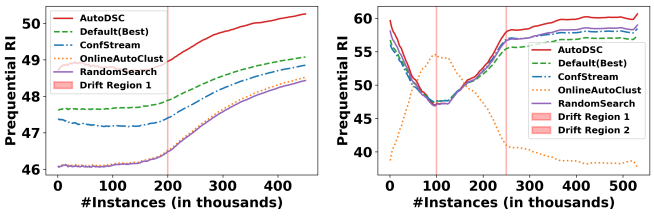
Table III presents a comprehensive comparison of AutoDSC against four baseline methods across twelve diverse datasets, evaluating both clustering quality (Rand Index) and computational efficiency (runtime).

AutoDSC achieves the highest overall average Rand Index of **61.321%**, substantially outperforming all baseline methods. The framework surpasses Default(Best) by 4.356 percentage points, ConfStream by 4.381 points, RandomSearch by 4.281 points, and OnlineAutoClust by 12.793 points. This demonstrates the effectiveness of the optimization approach in discovering optimal clustering configurations. The framework achieves the highest performance on 9 out of 12 datasets, with particularly notable improvements on complex synthetic datasets with concept drift, such as Moving RBF (75.786% vs. 66.393% for Default(Best)) and Moving Squares (71.514% vs. 64.806%). Temporal performance analysis reveals enhanced drift adaptation capabilities through rapid recovery after drift detection, as demonstrated in the SEA (Mixed) dataset (Figure 3a), and consistent outperformance across multiple drift regions in the Forest Cover dataset (Figure 3b). AutoDSC shows particularly effective performance on high-dimensional datasets like Forest Cover (54 attributes) and Vehicle (100 attributes) through multi-objective evaluation.

[1]Datasets and code: https://anonymous.4open.science/r/AutoDSC

TABLE II
SEARCH SPACE CONFIGURATION

Algorithm	Hyperparameter	Type	Range/Options
CluStream	n_macro_clusters	integer	[2, 10]
	max_micro_clusters	integer	[5, 50]
	time_window	integer	[100, 2000]
	micro_cluster_r_factor	numeric	[1.0, 5.0]
	time_gap	integer	[1, 100]
DenStream	decaying_factor	numeric	[0.01, 0.5]
	beta	numeric	[0.5, 1.0]
	mu	numeric	[2.0, 10.0]
	epsilon	numeric	[0.01, 0.5]
	n_samples_init	integer	[10, 1000]
	stream_speed	integer	[10, 1000]
DBSTREAM	clustering_threshold	numeric	[0.1, 2.0]
	fading_factor	numeric	[0.001, 0.1]
	cleanup_interval	integer	[1, 100]
	intersection_factor	numeric	[0.1, 0.9]
	minimum_weight	numeric	[0.5, 5.0]
KMeans	n_clusters	integer	[2, 30]
	halflife	numeric	[0.01, 1.0]
	mu	numeric	[0.0, 1.0]
	sigma	numeric	[0.1, 2.0]
	p	categorical	1, 2
STREAMKMeans	chunk_size	integer	[10, 100]
	n_clusters	integer	[2, 30]
	halflife	numeric	[0.01, 1.0]
	mu	numeric	[0.0, 1.0]
	sigma	numeric	[0.1, 2.0]
	p	categorical	1, 2



(a) SEA (Mixed) (b) Forest Cover

Fig. 3. Comparison of AutoDSC with baselines.

To rigorously validate these performance gains, the Critical Difference (CD) diagram in Figure 4 provides statistical validation of AutoDSC's performance using the Nemenyi post-hoc test with $\alpha = 0.10$. AutoDSC achieves the highest ranking with statistically significant improvements over all baseline methods, as evidenced by the absence of connecting lines between AutoDSC and these methods in the CD diagram. This comprehensive statistical validation confirms that AutoDSC's performance gains represent genuine algorithmic advances rather than random variation.

Beyond clustering quality, AutoDSC also demonstrates remarkable computational efficiency. While AutoDSC requires more computational resources than Default(Best), it achieves **2.5× faster execution** than ConfStream, **8.2× faster** than OnlineAutoClust, and **3.2× faster** than RandomSearch. This efficiency stems from AutoDSC's sampling ratio mechanism and efficient population management. Figure 5 illustrates the time-memory trade-off across methods. AutoDSC achieves an

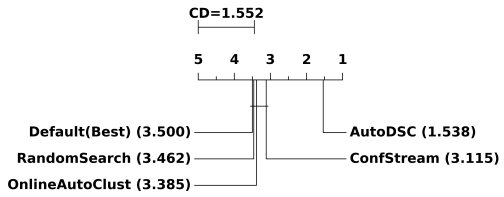


Fig. 4. Critical Difference diagram.

optimal balance, consuming moderate memory while maintaining reasonable execution times. In contrast, OnlineAutoClust exhibits the worst trade-off with both high execution time and substantial memory consumption, making it impractical for resource-constrained streaming environments.

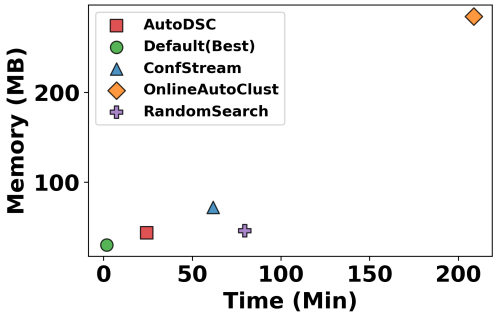


Fig. 5. Time-memory trade-off comparison.

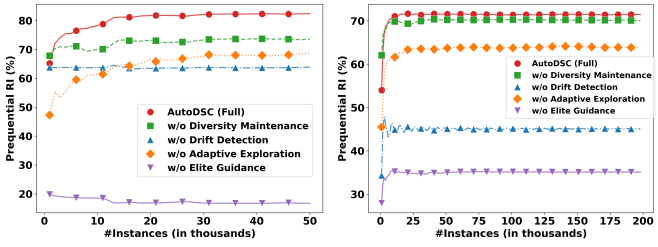
To further understand the contribution of individual components, we conducted a comprehensive ablation study across real-world (Insects) and synthetic (Moving Squares) datasets by systematically disabling individual mechanisms. Figure 6 demonstrates consistent patterns across both streams: elite guidance represents the most critical component, with its absence causing catastrophic performance degradation across both datasets. Diversity maintenance provides substantial performance retention compared to the full framework. Drift detection yields significant benefits with notable improvements across datasets, while adaptive exploration shows dynamic behavior with gradual improvement patterns. These findings establish elite guidance as essential while confirming that other components collectively enhance AutoDSC's robustness and validate the framework's integrated design.

Finally, beyond performance optimization, AutoDSC demonstrates intelligent adaptive configuration selection capabilities that dynamically adjust both clustering algorithms and their hyperparameter settings based on evolving stream characteristics. Figure 7 illustrates this comprehensive adaptive behavior on the Insects dataset, where the framework systematically transitions between different algorithms and hyperparameter combinations throughout the stream processing lifecycle.

TABLE III

CLUSTERING QUALITY ACROSS DATASETS, AVERAGED OVER 10 SEEDS. FOR EACH METHOD, ARI AND NMI ARE REPORTED (↑ HIGHER IS BETTER). BEST PER METRIC PER ROW IN **BOLD**, SECOND BEST UNDERLINED. METHODS GROUPED INTO AUTOML, THE RULE-BASED ADAPTIVE METHOD MOSTREAM, AND SINGLE (FIXED) ALGORITHMS. CONFSTR=CONFSTREAM, RANDS=RANDOMSEARCHSTREAM, OACLE=ONLINEAUTOCLUSTENSEMBLE, CLUSTR=CLUSTREAM, DENSTR=DENSTREAM.

Dataset	AutoML								Rule-based		Single algorithm							
	AutoDSC		ConfStr		RandS		OACIE		MOStr		KMeans		CluStr		DBSTR		DenStr	
	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI
Insects	0.066	0.107	0.016	0.035	0.054	0.113	0.048	0.212	0.090	**0.217**	0.032	0.061	**0.101**	0.150	0.000	0.007	0.000	0.000
KDD99	0.037	0.035	0.037	0.035	0.033	0.032	0.036	0.034	0.018	0.018	**0.038**	**0.036**	0.035	0.033	0.031	0.031	0.007	0.011
PenDigits	**0.446**	**0.615**	0.271	0.421	0.367	0.586	0.311	0.544	0.363	0.557	0.269	0.420	0.318	0.508	0.344	0.512	0.000	0.000
Shuttle	**0.354**	**0.477**	0.341	0.463	0.270	0.432	0.146	0.384	0.002	0.004	0.169	0.307	0.254	0.420	0.000	0.000	0.028	0.277
Vehicle	0.314	0.298	0.303	0.293	0.172	0.267	0.312	**0.312**	**0.335**	0.287	0.150	0.171	0.331	0.254	0.168	0.187	0.000	0.000
Blobs-Exp	**0.986**	**0.988**	0.953	0.965	0.844	0.899	0.895	0.893	0.938	0.964	0.855	0.894	0.817	0.902	0.000	0.000	0.228	0.630
Blobs-Grad	0.994	0.996	0.956	0.974	0.921	0.948	**0.995**	**0.997**	0.978	0.989	0.902	0.921	0.761	0.882	0.000	0.000	0.721	0.834
Blobs-MS	0.947	0.973	0.952	0.971	0.877	0.929	**0.985**	**0.988**	0.932	0.963	0.882	0.908	0.847	0.918	0.000	0.001	0.408	0.633
Blobs-Sud	**0.916**	**0.961**	0.904	0.952	0.878	0.931	0.855	0.934	0.774	0.893	0.892	0.940	0.812	0.912	0.000	0.000	0.638	0.823
RBF-Fast	0.993	0.996	0.988	0.987	0.924	0.954	**0.995**	**0.996**	0.911	0.958	0.795	0.888	0.811	0.908	0.000	0.001	0.735	0.840
Mean Score	**0.605**	**0.644**	0.572	0.610	0.534	0.609	0.558	0.629	0.534	0.585	0.498	0.555	0.509	0.589	0.054	0.074	0.276	0.405
Mean Rank	**1.90**	**1.90**	3.70	3.60	4.60	4.50	3.70	3.00	4.40	4.50	5.40	5.80	5.00	5.30	8.10	8.10	8.20	8.30



(a) Insects (b) Moving Squares

Fig. 6. Ablation study showing performance impact when individual AutoDSC components are disabled.

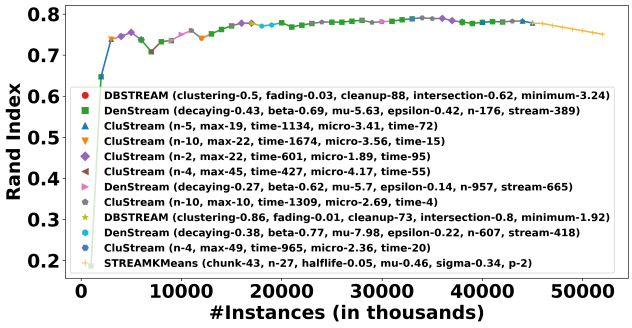


Fig. 7. Adaptive algorithm and hyperparameter evolution.

VII. DISCUSSION

AutoDSC represents a significant advancement in AutoML for streaming environments through several key innovations. Our multi-objective evaluation strategy, combining three normalized clustering validation indices, provides robust performance assessment compared to single-metric approaches used in ConfStream (Silhouette Width only) and OnlineAutoClust's ensemble voting. This prevents optimization convergence to configurations that excel in one aspect while failing catastrophically in others [33], [34]. The PGSSO outperforms existing methods by dynamically adjusting exploration rates based on performance trends, enabling rapid parameter space exploration after concept drift while efficiently exploiting successful configurations during stable periods, contrasting with ConfStream's fixed ensemble strategy and OnlineAutoClust's computationally intensive offline Bayesian optimization [25]. While our current evaluation focuses on tabular data streams with numerical and categorical features, extensions to high-dimensional sparse text streams, multi-modal event streams, or image-based data would require domain-specific clustering algorithms beyond this scope. These novel contributions collectively bridge the gap between traditional AutoML approaches and the dynamic requirements of deploying real-time streaming applications in IoT, sensor networks, and analytics platforms, where manual configuration is impractical [17].

VIII. CONCLUSION AND FUTURE WORK

We presented AutoDSC, a novel AutoML framework for adaptive data stream clustering that unifies algorithm selection, hyperparameter optimization, and concept drift adaptation. Extensive experiments across diverse datasets demonstrate AutoDSC's statistical significance, achieving the highest average Rand Index while executing faster than existing frameworks. The framework is supported by theoretical analysis providing convergence guarantees in non-stationary environments. Future research directions include extending AutoDSC to semi-supervised settings, multi-task clustering, and advanced multi-objective optimization techniques to balance trade-offs such as accuracy, efficiency, and robustness in dynamic settings.

REFERENCES

[1] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[2] Y. Poulakis, C. Doulkeridis, and D. Kyriazis, "A survey on automl methods and systems for clustering," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 5, pp. 1–30, 2024.

[3] D. Tschechlov, M. Fritz, and H. Schwarz, "Automl4clust: Efficient automl for clustering analyses," in *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021, pp. 343–348.

[4] Y. Liu, S. Li, and W. Tian, "Autocluster: Meta-learning based ensemble method for automated unsupervised clustering," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2021)*. Springer, 2021, pp. 246–258.

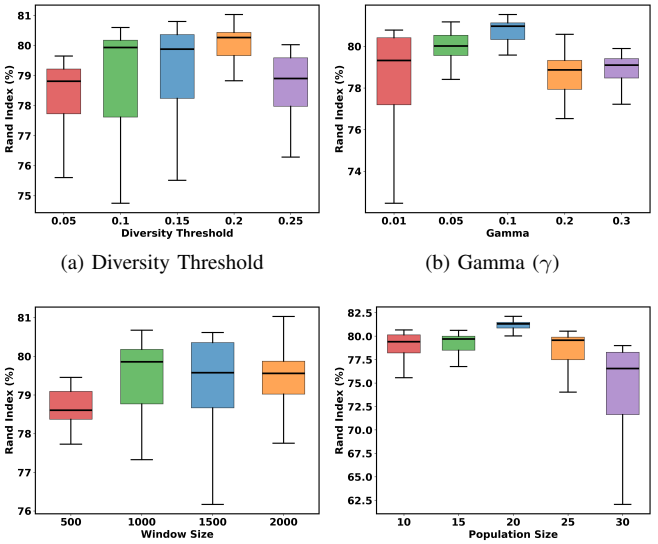
[5] R. ElShawi, H. Lekunze, and S. Sakr, "csmartml: A meta learning-based framework for automated selection and hyperparameter tuning for clustering," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 1119–1126.
[6] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "Ml2dac: Meta-learning to democratize automl for clustering analysis," *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3589289
[7] R. ElShawi and S. Sakr, "Tpe-autoclust: A tree-based pipline ensemble framework for automated clustering," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 1144–1153.
[8] M. C. da Silva, G. M. Tavares, E. Medvet, and S. B. Junior, "Problem-oriented automl in clustering," *arXiv preprint arXiv:2409.16218*, 2024, arXiv:2409.16218.
[9] L. Yang and A. Shami, "Iot data analytics in dynamic environments: From an automated machine learning perspective," *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105366, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197622003803
[10] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer, *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, 2018. [Online]. Available: https://moa.cms.waikato.ac.nz/book/
[11] B. Celik, P. Singh, and J. Vanschoren, "Online automl: An adaptive automl framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.
[12] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Asml: A scalable and efficient automl solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning*, ser. Proceedings of Machine Learning Research, K. Eggensperger, R. Garnett, J. Vanschoren, M. Lindauer, and J. R. Gardner, Eds., vol. 256. PMLR, 09–12 Sep 2024, pp. 11/1–26. [Online]. Available: https://proceedings.mlr.press/v256/verma24a.html
[13] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "Chacha for online automl," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 11 263–11 273. [Online]. Available: https://proceedings.mlr.press/v139/wu21d.html
[14] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Auto-reg: A dynamic automl framework for streaming regression," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2025, pp. 245–256.
[15] M. Carnein, H. Trautmann, A. Bifet, and B. Pfahringer, "confstream: Automated algorithm selection and configuration of stream clustering algorithms," in *International conference on learning and intelligent optimization*. Springer, 2020, pp. 80–95.
[16] R. El Shawi and D. Rozgonjuk, "Onlineautoclust: A framework for online automated clustering," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 3870–3874.
[17] M. Bahri, A. Bifet, J. Gama, H. M. Gomes, and S. Maniu, "Data stream analysis: Foundations, major tasks and tools," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 3, p. e1405, 2021.
[18] C. C. Aggarwal, "Data streams: models and algorithms," *Springer Science & Business Media*, 2013.
[19] C. C. Aggarwal, S. Y. Philip, J. Han, and J. Wang, "A framework for clustering evolving data streams," in *Proceedings 2003 VLDB conference*. Elsevier, 2003, pp. 81–92.
[20] F. Cao, M. Ester, W. Qian, and A. Zhou, "Density-based clustering over an evolving data stream with noise," in *SIAM International Conference on Data Mining (SDM)*, 2006. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972740.9
[21] M. Hahsler and M. Bolaños, "Clustering data streams based on shared density between micro-clusters," *IEEE Transactions on Knowledge and Data Engineering*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7425260
[22] D. Sculley, "Web-scale k-means clustering," in *WWW 2010, Raleigh, North Carolina, USA*, 2010. [Online]. Available: http://code.google.com/p/sofia-ml
[23] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem *et al.*, "River: machine learning for streaming data in python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
[24] L. O'Callaghan, N. Mishra, A. Meyerson, S. Guha, and R. Motwani, "Streaming-data algorithms for high-quality clustering," in *Proceedings of the 18th International Conference on Data Engineering (ICDE'02)*, 2002. [Online]. Available: http://www-db.stanford.edu/~loc/icde02.ps
[25] F. Hinder, A. Artelt, and B. Hammer, "Towards non-parametric drift detection via dynamic adapting window independence drift detection (dawidd)," in *International Conference on Machine Learning*. PMLR, 2020, pp. 4249–4259.
[26] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine learning*, vol. 47, no. 2-3, pp. 235–256, 2002.
[27] N. Cesa-Bianchi and G. Lugosi, *Prediction, learning, and games*. Cambridge university press, 2006.
[28] O. Besbes, Y. Gur, and A. Zeevi, "Stochastic multi-armed-bandit problem with non-stationary rewards," in *Advances in neural information processing systems*, 2014, pp. 199–207.
[29] W. M. Rand, "Objective criteria for the evaluation of clustering methods," *Journal of the American Statistical association*, vol. 66, no. 336, pp. 846–850, 1971.
[30] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.
[31] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *The journal of machine learning research*, vol. 13, no. 1, pp. 281–305, 2012.
[32] P. Gijsbers, M. L. Bueno, S. Coors, E. LeDell, S. Poirier, J. Thomas, B. Bischl, and J. Vanschoren, "Amlb: an automl benchmark," *Journal of Machine Learning Research*, vol. 25, no. 101, pp. 1–65, 2024.
[33] O. Arbelaitz, I. Gurrutxaga, J. Muguerza, J. M. Pérez, and I. Perona, "An extensive comparative study of cluster validity indices," *Pattern recognition*, vol. 46, no. 1, pp. 243–256, 2013.
[34] Y. Liu, Z. Li, H. Xiong, X. Gao, and J. Wu, "Understanding of internal clustering validation measures," in *2010 IEEE International Conference on Data Mining*, 2010, pp. 911–916.

APPENDIX

A. Sensitivity Analysis

Comprehensive sensitivity analysis conducted across all twelve datasets evaluated AutoDSC's robustness across systematic hyperparameter variations, establishing optimal configuration ranges essential for practical deployment in diverse streaming environments. Results are averaged across real-world and synthetic streams to ensure generalizability. The diversity threshold parameter exhibits optimal performance within the range $\tau_{\text{div}} = 0.15 - 0.25$, achieving peak clustering quality (Figure 8a), with lower threshold values causing premature convergence by allowing similar configurations to dominate the population, while higher values over-restrict configuration selection and reduce optimization efficiency. Adaptation rate analysis reveals $\gamma = 0.1$ as the optimal setting (Figure 8b), as lower adaptation rates, such as $\gamma = 0.01$, result in sluggish responses to changing performance trends. In comparison, higher rates, including $\gamma \geq 0.2$ introduce excessive volatility that destabilizes the optimization process. Window size evaluation identifies the optimal range as $w = 1000 - 1500$, achieving consistent performance levels (Figure 8c), with smaller window sizes providing insufficient data for reliable clustering evaluation and introducing noise through excessive optimization frequency, while larger windows delay adaptation responses to concept drift events. Population size demonstrates the most dramatic performance variation across tested values, with a clear optimum at $K = 20$ (Figure 8d), as smaller population sizes limit exploration coverage of the parameter space, while larger sizes introduce computational overhead and complicate elite population management. These

findings establish robust optimal configuration guidelines for practical AutoDSC deployment: diversity threshold $\tau_{\text{div}} = 0.1$, adaptation rate $\gamma = 0.1$, window size $w = 1000$, and population size $K = 20$, demonstrating that AutoDSC exhibits stable performance within reasonable hyperparameter ranges while showing graceful degradation outside optimal regions, with population size representing the most critical parameter requiring careful tuning for optimal performance.



(a) Diversity Threshold (b) Gamma (γ)

(c) Window Size (d) Population Size

Fig. 8. Sensitivity analysis revealing optimal hyperparameter ranges for AutoDSC deployment.

B. Adaptive Exploration Rate Dynamics

Figure 9 demonstrates the effectiveness of AutoDSC's adaptive exploration rate mechanism through real-time performance tracking over 50 processing windows. The exploration rate (blue line) exhibits intelligent adaptive behavior, beginning at a high value of 0.50 to ensure comprehensive initial search of the configuration space, then systematically decreasing as the composite clustering score (red line) rapidly improves from 0.52 to 0.80 within the first three windows. This dynamic adjustment, controlled by the γ parameter in the AutoDSC Framework, exemplifies the Population-Guided Stochastic Search Optimizer's ability to automatically transition from exploration to exploitation phases. Once optimal performance is achieved, the exploration rate stabilizes around 0.477, maintaining sufficient exploration to handle potential concept drift while primarily exploiting the discovered high-quality configurations. The sustained composite score of approximately 0.80 throughout the remaining windows validates the framework's capacity to maintain clustering quality while adapting its search strategy, demonstrating that AutoDSC successfully learns from elite configurations and avoids the inefficiencies associated with fixed exploration strategies.

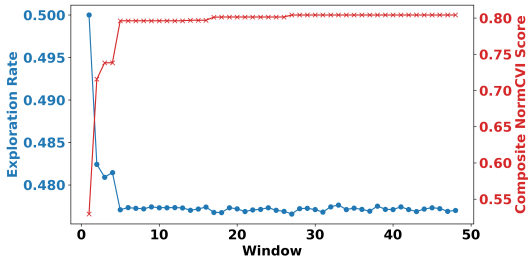


Fig. 9. Adaptive exploration rate dynamics showing the evolution of exploration rate (blue) and composite clustering score (red) over the windows.

C. Performance Variability Analysis

Comprehensive performance variability analysis across 10 independent runs with different random seeds reveals AutoDSC's optimal balance between clustering effectiveness and result consistency in the performance-variance trade-off landscape. Figure 10 demonstrates that AutoDSC achieves superior performance with 61.5% average Rand Index while maintaining reasonable variance of 0.4%, establishing the most desirable combination of clustering quality and stability compared to all baseline methods.

The Default(Best) configuration exhibits the lowest variance at 0.1% but suffers from substantially lower performance at 57.2% Rand Index, representing a conservative approach that prioritizes consistency over clustering effectiveness. ConfStream and RandomSearch demonstrate similar moderate performance around 57.0–57.2% with comparable variance levels of approximately 0.4%, indicating consistent but suboptimal clustering capabilities. OnlineAutoClust presents the most problematic performance profile, combining poor clustering quality (48.5% Rand Index) with extremely high variance (2.0%), suggesting fundamental instability that makes it unreliable for practical deployment where consistent results are essential.

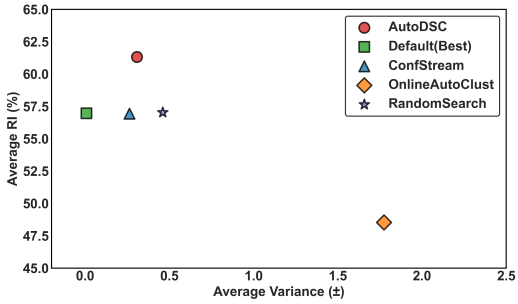


Fig. 10. Performance-variance trade-off analysis showing average Rand Index performance versus standard deviation across all datasets and methods.

D. Memory Usage Analysis

Memory efficiency represents a critical performance dimension for streaming algorithms operating under resource constraints in real-world deployment scenarios. Table IV presents a comprehensive memory consumption analysis across twelve

diverse datasets, revealing AutoDSC's competitive resource utilization characteristics.

E. Drift Detection Method Selection

We adopt the Dynamic Adapting Window Independence Drift Detection (DAWIDD) method [25] for detecting concept drift in AutoDSC. Unlike traditional loss-based drift detectors such as DDM, EDDM, and ADWIN that rely on classification error as a proxy for distributional change, DAWIDD provides a non-parametric, model-free approach by reformulating drift detection as an independence test between data observations and their timestamps. This theoretical foundation, grounded in the equivalence between concept drift and statistical dependency of random variables, enables DAWIDD to detect diverse drift characteristics including both gradual and abrupt changes without requiring assumptions about the underlying drift type. As shown in Figure 11, DAWIDD consistently achieves superior True Positive (TP) detection rates on benchmark datasets common to both studies: obtaining TP values of 1.70 on Hyperplane, 1.30 on Random (RBF), and 1.40 on Forest Cover—substantially outperforming HDDDM, EDDM, DDM, and ADWIN which frequently achieve TP values below 0.5 on these same datasets. This superior detection capability, combined with its model-free nature, makes DAWIDD particularly well-suited for AutoDSC where the underlying clustering model is automatically selected and drift characteristics are unknown a priori.

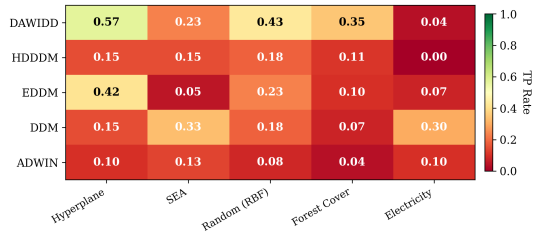


Fig. 11. True Positive (TP) rates comparison across drift detection methods. DAWIDD demonstrates consistently superior detection performance across benchmark datasets, validating its selection for AutoDSC.

TABLE IV
MEMORY USAGE COMPARISON (MB), BEST PERFORMANCE IS HIGHLIGHTED IN **BOLD**, SECOND-BEST IS UNDERLINED.

Dataset	AutoDSC	Default(Best)	ConfStream	OnlineAutoClust	RandomSearch
Adult	8.766	**0.821**	8.184	35.373	3.090
Electricity	9.183	**0.778**	6.541	35.891	7.646
Forest Cover	237.523	244.101	224.018	1801.302	**38.135**
Hyperplane	**37.868**	45.298	43.528	297.227	139.636
Insects	92.183	13.167	**1.122**	123.561	1.262
Moving RBF	20.298	15.037	**2.240**	156.038	12.325
Moving Squares	25.253	**0.000**	7.554	99.255	7.175
New Airlines	6.042	**0.433**	623.513	282.372	169.783
Sea High Abrupt Drift	7.271	11.384	**4.536**	178.975	90.451
Sea High Mixed Drift	8.119	10.974	**3.887**	188.485	93.002
Synth RandomRBF Drift	80.109	**1.746**	8.683	72.791	7.795
Vehicle SensIT	69.878	74.954	56.573	595.520	**50.957**
Mean	50.208	**34.891**	82.532	322.233	51.772